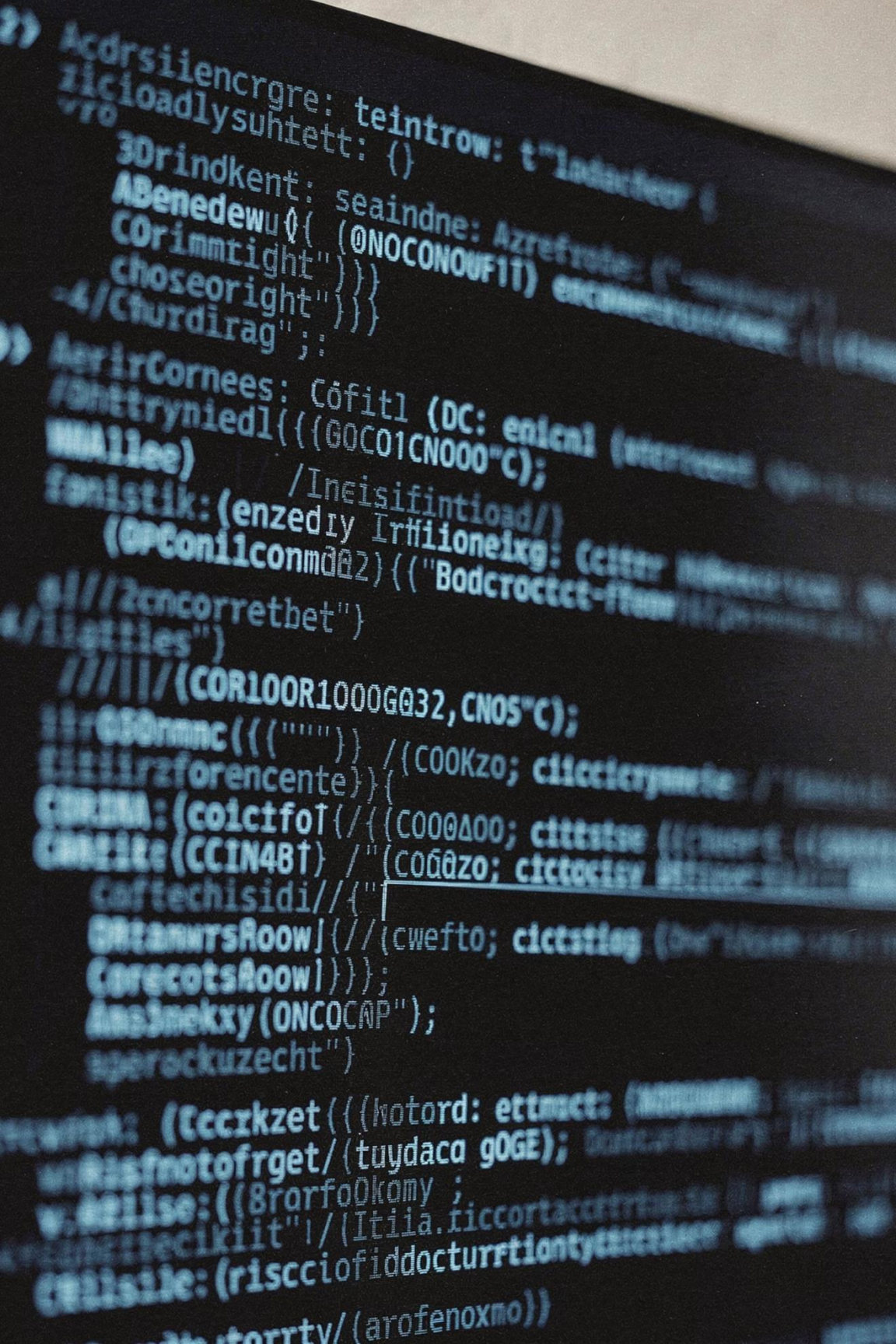


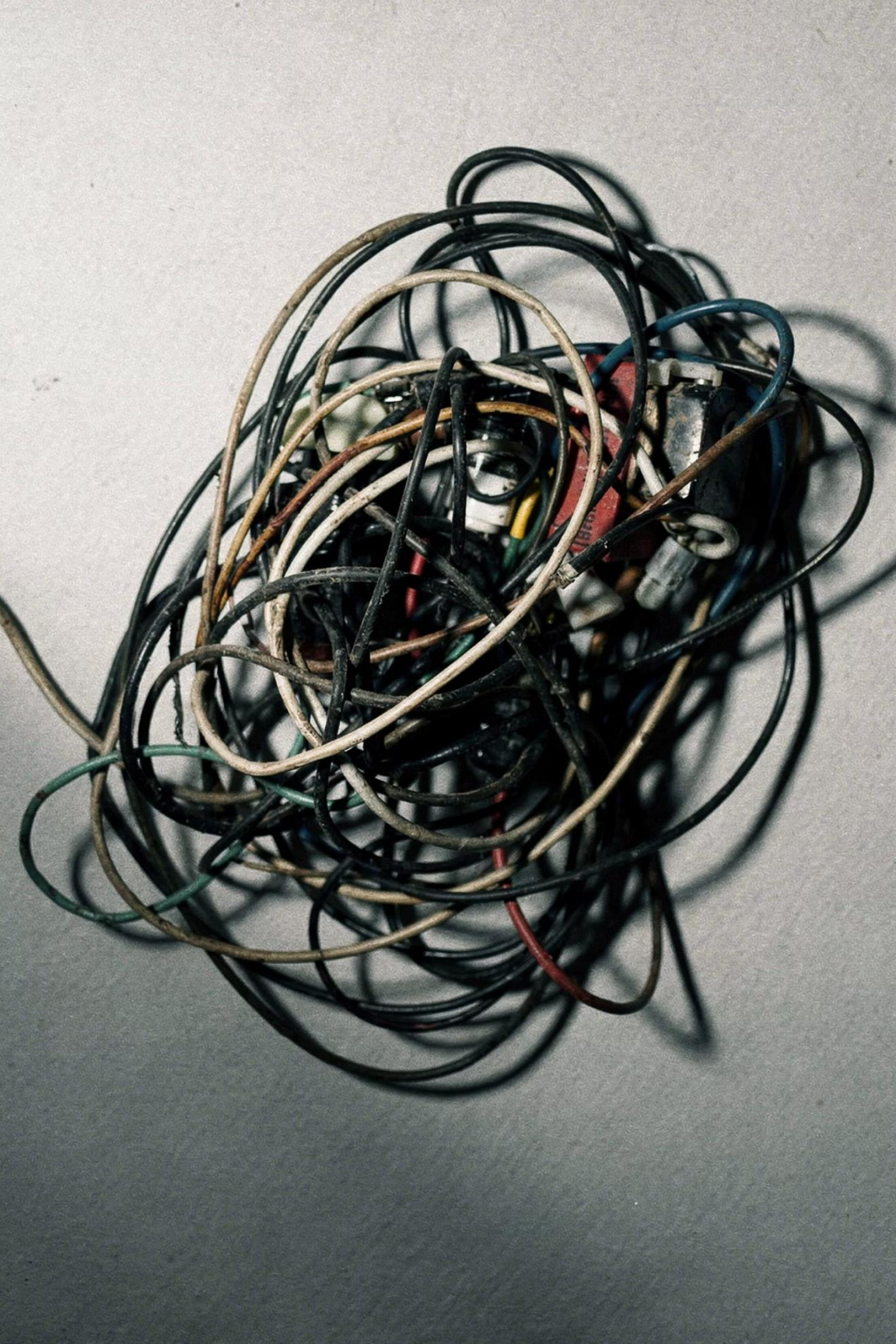
Angular Moderno

24 - Estado Global com Signals + Service



24 – Estado Global com Signals + Service

Objetivo da aula: Nesta aula vamos evoluir a arquitetura da aplicação Angular moderna, resolvendo o problema de estado espalhado e centralizando o estado do carrinho em um service global.



Problema Atual — Estado Espalhado

Atualmente, o estado do carrinho está **dentro do componente de produtos**. Isso gera uma série de problemas que impedem a escalabilidade da aplicação.

Outros componentes não acessam o carrinho

O estado fica preso dentro de um único componente, sem possibilidade de compartilhamento.

Header não conhece o estado

O cabeçalho da aplicação não tem acesso ao número de itens no carrinho.

Página de carrinho isolada

A página de carrinho não utiliza o mesmo estado, gerando inconsistências.

Código não reutilizável

A lógica duplicada dificulta manutenção e evolução do projeto.



Precisamos de um **estado global** para resolver esses problemas.

O que é um Service no Angular Moderno

Um **Service** é uma classe responsável por centralizar dados, compartilhar estado e reutilizar lógica, funcionando como a **fonte única da verdade** da aplicação.

Responsabilidades do Service

- Centralizar dados
Um único lugar para todos os dados da aplicação.
- Compartilhar estado
Qualquer componente pode acessar o mesmo estado.
- Reutilizar lógica
Evita duplicação de código entre componentes.
- Fonte única da verdade
Garante consistência em toda a aplicação.

Relação com o Projeto

Components

Exibem dados

Services

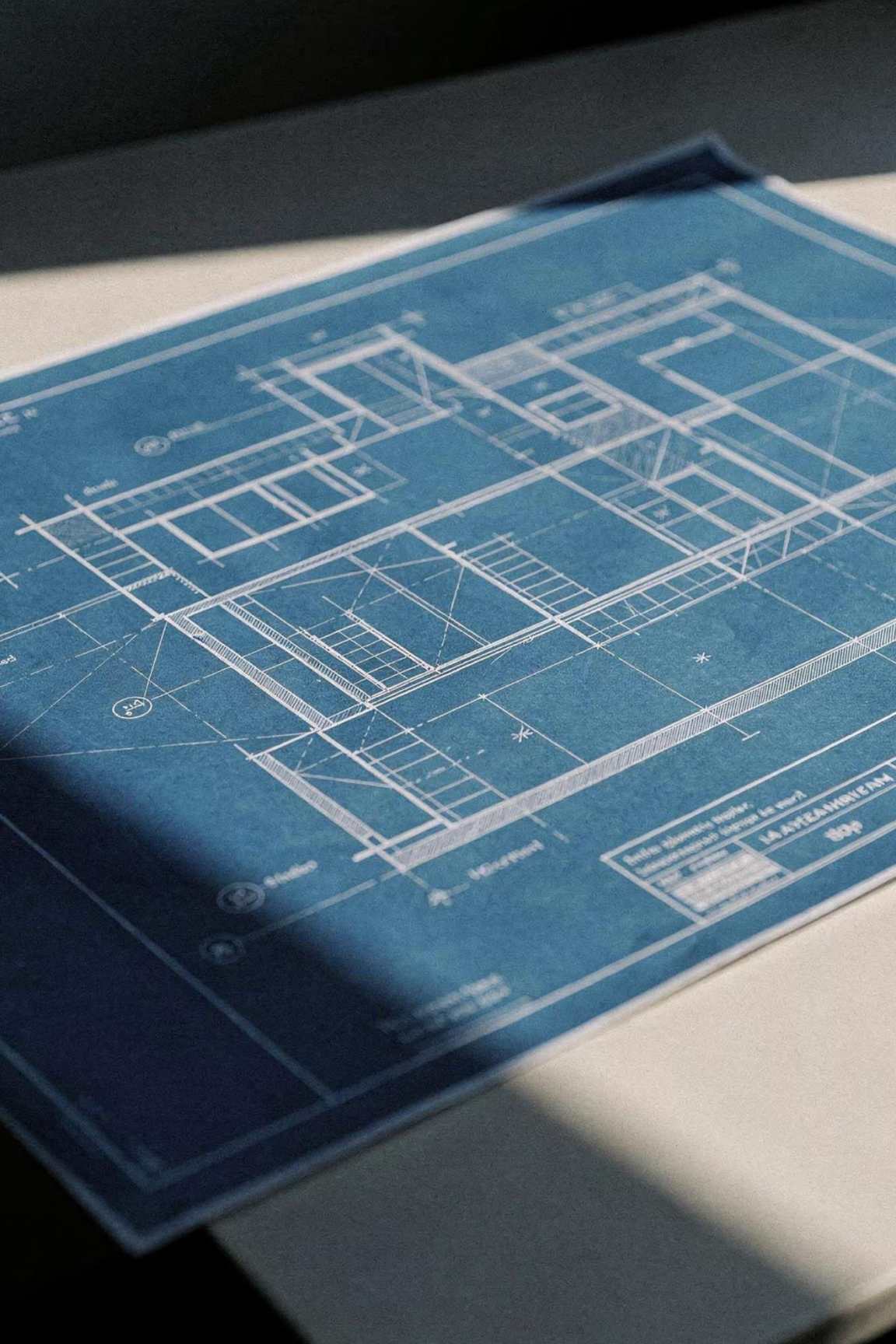
Controlam dados

Core

Serviços globais

Features

Telas da aplicação



Organização da Arquitetura

A separação de responsabilidades deixa o projeto **escalável e organizado**. Cada pasta tem um papel bem definido na estrutura da aplicação.



core

Serviços globais: autenticação, API e estado compartilhado entre toda a aplicação.



features

Telas da aplicação: produtos, carrinho, home — cada feature isolada em seu módulo.



shared

Componentes reutilizáveis que podem ser usados em qualquer parte da aplicação.



interceptors

Interceptação HTTP para tratamento centralizado de requisições e respostas.

Passo 1 — Criar Pasta de Serviços

Estrutura de Pastas

```
src/  
└─ app/  
    └─ core/  
        └─ services/ ← criar aqui
```

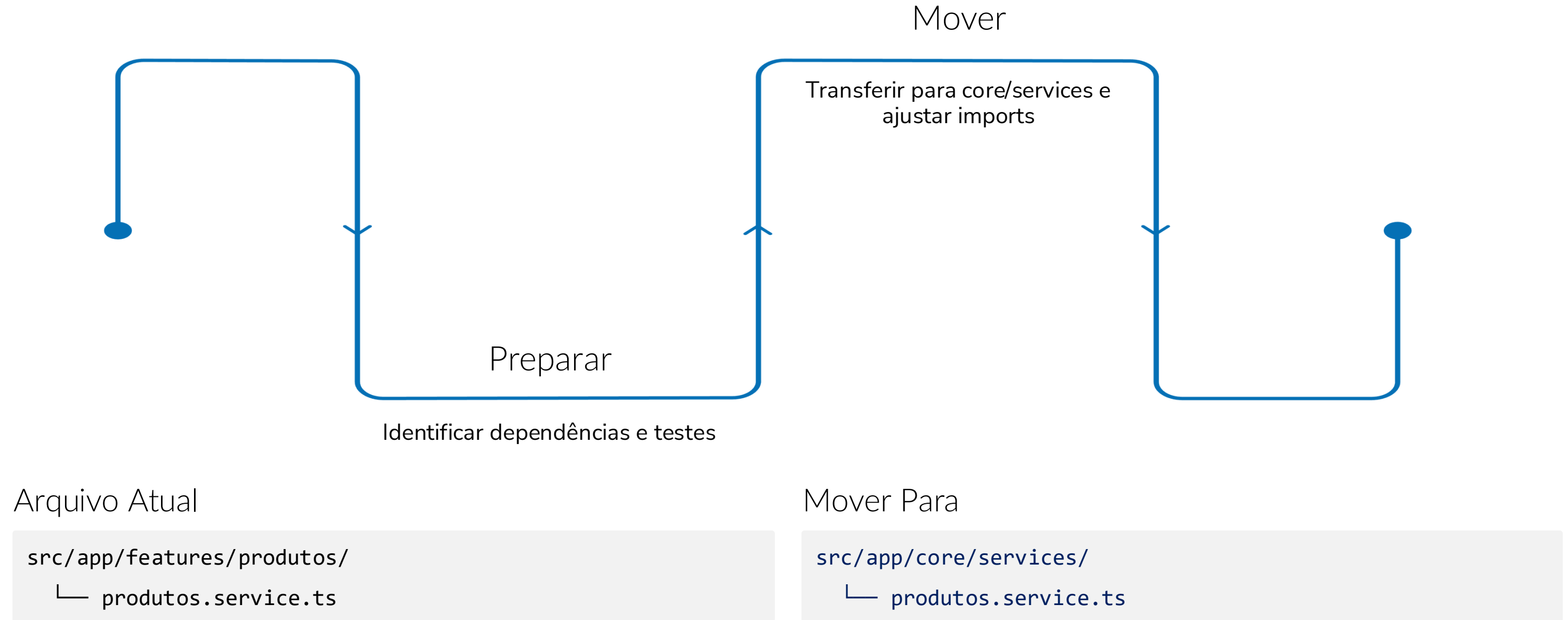
Esta pasta centraliza todos os serviços globais da aplicação.

Por que centralizar?

- 1 Organização clara
Todos os serviços globais em um único lugar previsível.
- 2 Fácil localização
Qualquer desenvolvedor sabe onde encontrar os serviços.
- 3 Escalabilidade
Novos serviços são adicionados sem bagunçar a estrutura.

Passo 2 – Mover ProdutosService

O `ProdutosService` consome a API e pode ser usado em várias partes da aplicação. Por isso, deve estar no **core**, e não dentro de uma feature específica.



O conteúdo do arquivo `produtos.service.ts` permanece **exatamente igual**. Apenas o local muda.

Passo 3 — Corrigir Import do ProdutosService

Após mover o arquivo, o caminho do import precisa ser atualizado no componente que o utiliza.

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

ANTES

```
import { ProdutosService } from '../produtos.service';
```

DEPOIS

```
import { ProdutosService } from  
'../../../../core/services/produtos.service';
```

 O caminho relativo `../../../../` sobe três níveis na hierarquia de pastas até chegar ao diretório `core/services`.

Passo 4 — Criar CarrinhoService

Agora vamos criar o serviço responsável pelo **estado global do carrinho**. Este é o coração da nossa refatoração.

Arquivo a Criar

```
src/app/core/services/  
└─ carrinho.service.ts
```

Este service será a **fonte única da verdade** para o estado do carrinho em toda a aplicação.

O que este service fará?

- Armazenar itens do carrinho

Usando Signals para reatividade automática.
- Calcular totais

Quantidade e valor total derivados do estado.
- Expor ações

Adicionar e limpar itens do carrinho.

```
import { Injectable, signal, computed } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class CarrinhoService {  
  // STATE (GLOBAL)  
  private carrinho = signal<{ nome: string; preco: number }[]>([]);  
  
  // SELECTORS  
  itens = computed(() => this.carrinho());  
  quantidade = computed(() => this.carrinho().length);  
  total = computed(() =>  
    this.carrinho().reduce((total, item) => total + item.preco, 0)  
  );  
  
  // ACTIONS  
  adicionar(produto: { nome: string; preco: number }) {  
    this.carrinho.update(lista => [...lista, produto]);  
  }  
  limpar() {  
    this.carrinho.set([]);  
  }  
}
```

Estrutura do CarrinhoService

O `CarrinhoService` segue o padrão moderno de gerenciamento de estado com **Signals**, separando claramente estado, seletores e ações.



State — `signal()`

carrinho: o array de itens do carrinho, armazenado como um Signal reativo que notifica automaticamente os consumidores.



Selectors — `computed()`

itens, quantidade e total: dados derivados do estado principal, recalculados automaticamente quando o estado muda.



Actions — métodos

adicionar e limpar: funções que modificam o estado de forma controlada e previsível.



Passo 5 — Atualizar ListaProdutos (Parte 1 de 3)

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

Parte 1 - Remove estado local

O array local de carrinho dentro do componente é removido completamente.

REMOVER estas linhas:

```
carrinho = signal<{ nome: string; preco: number }[]>([]);

quantidadeCarrinho = computed(() => this.carrinho().length);

totalCarrinho = computed(() => {
  return this.carrinho().reduce((total, item) => total + item.preco, 0);
});
```

Passo 5 — Atualizar ListaProdutos (Parte 2 de 3)

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

Parte 2 - Usa CarrinhoService

Injeta o `CarrinhoService` e delega todas as operações de carrinho para ele.

Injetar o Service:

```
// Junto aos imports no início do arquivo
import { CarrinhoService } from '../../core/services/carrinho.service';

// Dentro da classe ListaProdutos
carrinhoService = inject(CarrinhoService);
```

Passo 5 — Atualizar ListaProdutos (Parte 3 de 3)

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

Parte 3 - Centraliza lógica

O componente passa a ser responsável apenas pela exibição, não pelo estado.

Substituir o método `adicionarAoCarrinho()`:

```
adicionarAoCarrinho(produto: { nome: string; preco: number }) {  
  this.carrinhoService.adicionar(produto);  
}
```

Abaixo dos injects de `produtosService` e `carrinhoService`:

```
quantidadeCarrinho = this.carrinhoService.quantidade;  
totalCarrinho = this.carrinhoService.total;
```

✅ Princípio aplicado: **Single Responsibility**. Cada classe tem uma única responsabilidade bem definida.

Passo 6 — Atualizar Template de Produtos

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.html`

O que muda no template?

→ Referência ao service

O template passa a referenciar `carrinhoService` diretamente para exibir dados reativos.

→ Bindings atualizados

Os event bindings e property bindings apontam para os métodos e signals do service.

→ Reatividade automática

Com Signals, o template atualiza automaticamente sem necessidade de `ChangeDetection` manual.

Mude toda a seção do carrinho para esta:

```
<!-- Seção do Carrinho -->
<section>
  <hr />
  <h2>Carrinho:</h2>

  @for (item of carrinhoService.itens(); track $index) {
    <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
  }

  <h3>Itens no carrinho: {{ carrinhoService.quantidade() }}</h3>
  <h3>Total do Carrinho: R$ {{ carrinhoService.total().toFixed(2) }}</h3>
</section>
```

Passo 7 — Compartilhamento Real do Estado

Arquivo: `src/app/shared/layout/header/header.ts`

Fazendo o carrinho funcionar no header. Veja como ficou o código completo deste arquivo:

```
import { Component, inject } from '@angular/core';
import { RouterLink } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';

import { CarrinhoService } from '../../../core/services/carrinho.service';

@Component({
  selector: 'app-header',
  imports: [MatToolbarModule, MatButtonModule, RouterLink],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {
  private carrinhoService = inject(CarrinhoService);
  quantidade = this.carrinhoService.quantidade;
}
```

Passo 8 – Atualizar Header HTML

Arquivo: `src/app/shared/layout/header/header.html`

O Header agora exibe a **quantidade de itens do carrinho em tempo real**, consumindo diretamente o `CarrinhoService`.

```
<mat-toolbar color="primary">
  <span>Minha Loja</span>

  <span class="flex-spacer"></span>

  <button mat-button routerLink="/produtos">Produtos</button>
  <button mat-button routerLink="/carrinho">Carrinho ({{ quantidade() }})</button>
</mat-toolbar>
```

1

Injetar `CarrinhoService`

O Header passa a depender do `CarrinhoService` para acessar o estado global do carrinho.

2

Exibir badge reativo

Um badge numérico exibe `carrinhoService.quantidade()`, atualizado automaticamente via Signal.

3

Navegação com `RouterLink`

O link para a página do carrinho usa a diretiva `RouterLink` no padrão standalone moderno. Observe o número ao lado do link do Carrinho.

Passo 9 — Executar a Aplicação

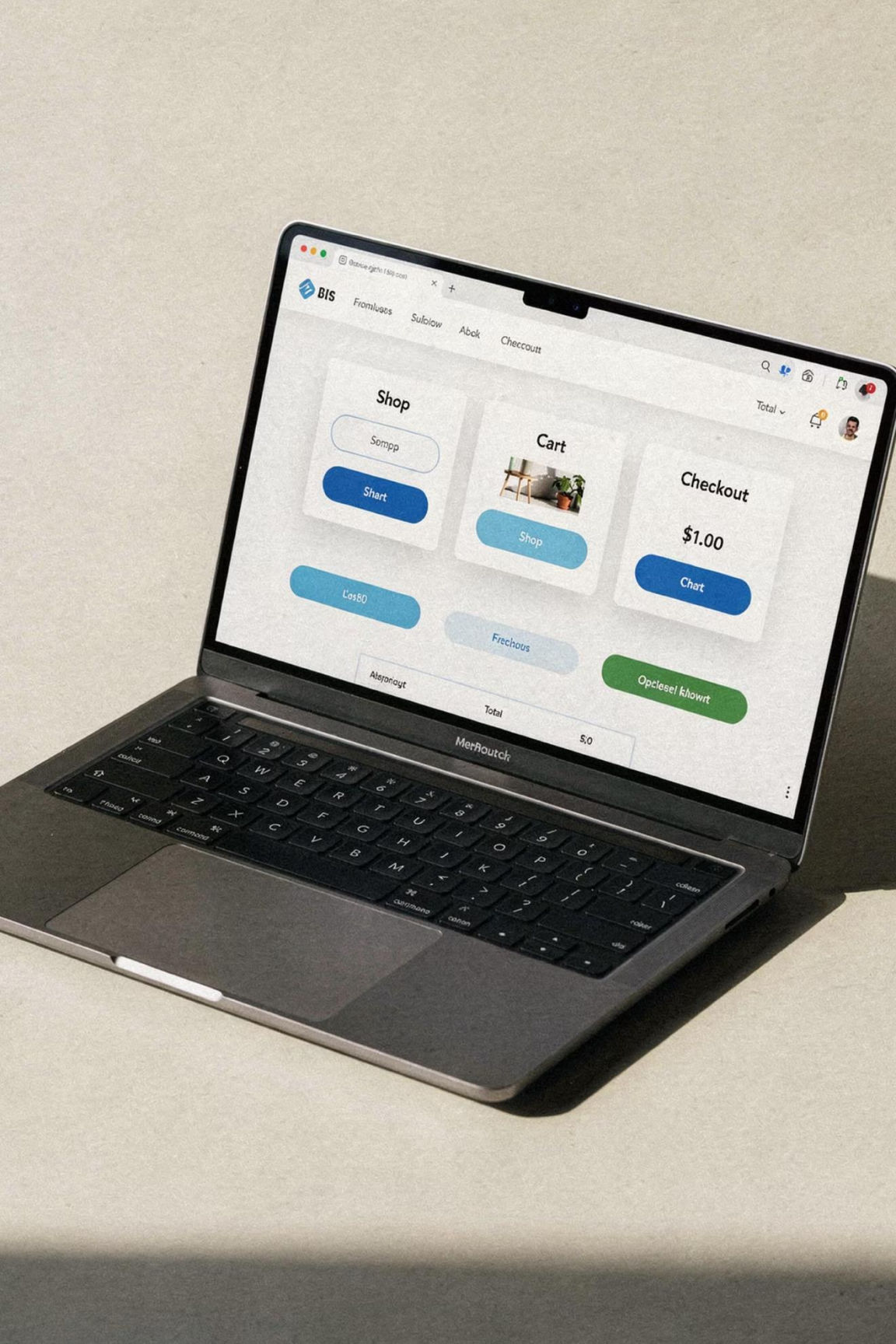
Comando

```
ng serve
```

Execute no terminal, na raiz do projeto Angular.

Acesse em: <http://localhost:4200>





Resultado Esperado

Ao acessar a aplicação após as alterações, o comportamento esperado demonstra que o **estado global está funcionando corretamente**.



Adicionar produtos

Clicar no botão de adicionar produto funciona normalmente na lista de produtos.



Número no Header

O contador no cabeçalho exibe imediatamente a quantidade de itens adicionados.



Atualização automática

O estado é atualizado em tempo real graças à reatividade dos Signals.



Sem recarregar a página

Nenhum reload necessário — a interface reage instantaneamente às mudanças de estado.



Estado global funcionando corretamente! Observe o número ao lado do link Carrinho!

Observação Importante

O carrinho ainda aparece na tela de produtos

Você pode notar que o resumo do carrinho ainda está visível na tela de lista de produtos. **Isso é completamente intencional** neste momento da aula.

Por que mantemos assim?

Validação do estado global

Manter o carrinho visível na tela de produtos nos permite confirmar visualmente que o estado global está sendo atualizado corretamente.

Foco no aprendizado

O objetivo desta aula é criar e validar o estado global. A organização visual das telas virá nas próximas etapas.

Lembre-se

Em desenvolvimento, é comum manter elementos temporários para facilitar a validação antes de refatorar a interface final.

 A tela de carrinho será implementada nas próximas aulas!

Próxima Evolução

Criamos a base do estado global hoje. Nas próximas aulas, vamos **consumir esse estado** nas telas corretas da aplicação.



Hoje — Estado criado

Criamos o `CarrinhoService` com Signals, centralizamos o estado e conectamos o Header.



Próximas aulas — Mover o carrinho

O carrinho será removido da tela de produtos e movido para a tela correta da aplicação.



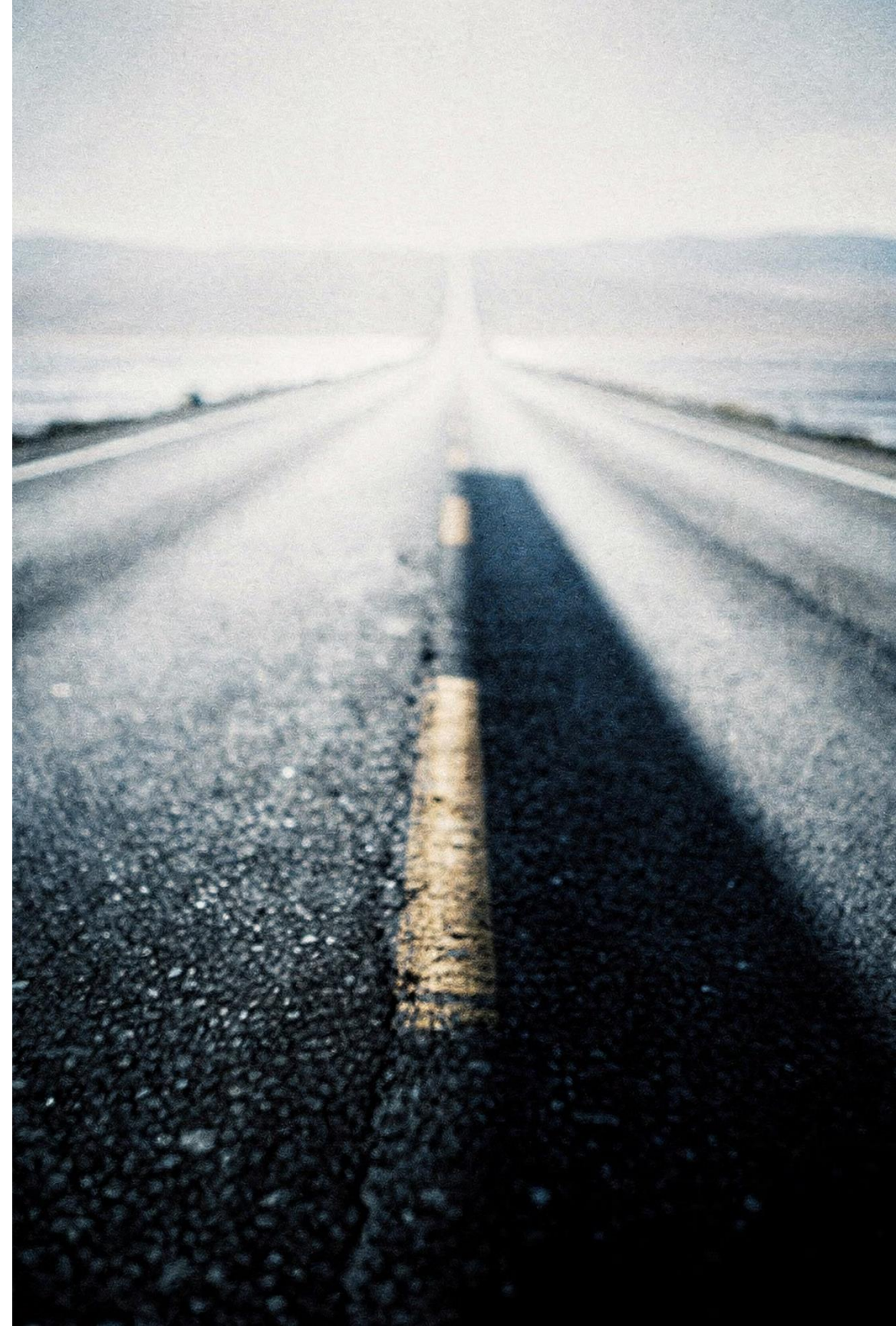
Feature carrinho implementada

A feature completa de carrinho será implementada com sua própria tela e componentes.



Estado consumido pela página

A página de carrinho consumirá o estado global criado hoje, exibindo todos os itens adicionados.



Conclusão da Aula

Nesta aula evoluímos significativamente a arquitetura da aplicação Angular, aplicando conceitos fundamentais de desenvolvimento profissional.

Estado Global com Signals

Aprendemos a usar `signal()` e `computed()` para criar estado reativo e compartilhado entre componentes.

Service como Fonte Única

O `CarrinhoService` centraliza toda a lógica do carrinho, eliminando duplicação e inconsistências.

Compartilhamento entre Componentes

Header, lista de produtos e futura página de carrinho compartilham o mesmo estado de forma transparente.

Organização Arquitetural Moderna

Estrutura `core/features/shared` garante escalabilidade e manutenibilidade do projeto.

Mensagem final: Separar estado da interface é essencial para aplicações profissionais. Um componente deve saber *exibir*. Um service deve saber *controlar*.

Dúvidas

?

